Liquidsoap

Liquidsoap is a powerful and flexible language for describing audio and video streams. It offers a rich collection of operators that you can combine at will for creating or transforming streams.

Liquidsoap in AzuraCast

AzuraCast uses Liquidsoap as the the AutoDJ to generate and manipulate audio streams. It transforms the input streams to the correct format for each mount point, handles live DJ connections, filtering, compression, and more.

Liquidsoap Annotation Reference

This information is intended for plugin developers who are looking to supply additional annotation metadata to Liquidsoap. It is a compilation of information that wasn't available elsewhere online, and is intended only for advanced users.

Appending Tracks

> liquidsoap/src/operators/append.ml

Append an extra track to every track.

liq_append

Set the metadata 'liq_append' to 'false' to inhibit appending on one track.

Crossing Tracks

> liquidsoap/src/operators/cross.ml

Common effects like cross-fading can be split into two parts: crossing, and fading. Here we implement crossing, not caring about fading: a arbitrary transition function is passed, taking care of the combination.

A buffer is needed to store the end of a track before combining it with the next track. We could always have a full buffer, but this would involve copying all the time. Instead, we try to fill the buffer only when getting close to the end of track. The problem then is to cope with tracks which are longer than expected, i.e. which end doesn't really fit in the buffer.

This operator works with any type of stream. All three parameters are durations in ticks.

liq_start_next

Metadata field which, if present and containing a float, overrides the 'duration' parameter for current track.

Cue Points

> liquidsoap/src/operators/cuepoint.ml

The [cue_cut] class is able to skip over the beginning and end of a track according to cue points. This involves quite a bit of trickery involving clocks, #seek as well as reverting frame contents. Even more trickery would be needed to implement a [cue_split] operator that splits tracks according to cue points: in particular, the frame manipulation would get nasty, involving storing chunks that have been fetched too early, replaying them later, glued with new content.

We use ticks for precision, but store them as Int64 to allow long durations. This should eventually be generalized to all of liquidsoap, removing limitations such as the duration

passed to #seek or returned by #remaining. We introduce a few notations to make this comfortable.

Start track after a cue in point and stop it at cue out point. The cue points are given as metadata, in seconds from the begining of tracks.

liq_cue_in

Metadata for cue in points.

liq_cue_out

Metadata for cue out points. Note that cue-out should be given relative to the beginning of the file (0:00 of the file itself, not 0:00 as calculated by the cue-in point).

Fader

> liquidsoap/src/operators/fade.ml

Fade durations (in, initial, out, final) are indicated in total seconds.

liq_fade_type

Metadata field which, if present and correct, overrides the 'type' parameter for current track.

Options: lin|sin|log|exp (linear, sinusoidal, logarithmic or exponential)

Default: lin

liq_fade_in

Fade the beginning of **tracks**.

liq_fade_initial

Fade the beginning of **a stream**.

liq_fade_out

Fade the end of **tracks**.

liq_fade_final

Fade **a stream** to silence.

Offset

> liquidsoap/src/operators/on_offset.ml

Call a given handler when position in track is equal or more than a given amount of time (the ‘offset’ parameter)

liq_on_offset

Metadata field which, if present and containing a float, overrides the ‘offset’ parameter.

Prepending Tracks

> liquidsoap/src/operators/prepend.ml

Prepend an extra track before every track.

liq_prepend

Set the metadata ‘liq_prepend’ to ‘false’ to inhibit appending on one track.

Set Volume

> liquidsoap/src/operators/setvol.ml

Multiply the amplitude of the signal.

liq_amplify

Specify the name of a metadata field that, when present and well-formed, overrides the amplification factor for the current track. Well-formed values are floats in decimal notation (e.g. '0.7') which are taken as normal/linear multiplicative factors; values can be passed in decibels with the suffix 'dB' (e.g. '-8.2 dB', but the spaces do not matter).

Video Fade

> liquidsoap/src/operators/video_fade.ml

liq_video_fade_in

Fade the beginning of tracks. Metadata 'liq_video_fade_in' can be used to set the duration for a specific track (float in seconds).

liq_video_fade_out

Fade the end of tracks. Metadata 'liq_video_fade_out' can be used to set the duration for a specific track (float in seconds).

LADSPA Plugins

LADSPA is a standard that allows software audio processors and effects to be plugged into a wide range of audio synthesis and recording packages.

Our latest rolling-release Docker image and Ansible scripts add support for LADSPA to Liquidsoap. We include several popular plugins, allowing you to perform powerful equalization in your configuration.

Use the following command to view all the available LADSPA plugins that come pre-installed with AzuraCast:

```
docker-compose exec --user=azuracast web liquidsoap --list-plugins
```

To get detailed information about the usage of a specific LADSPA plugin use the following command and replace the `plugin` part of the `ladspa.plugin` with the name of the plugin you want to look at:

```
docker-compose exec --user=azuracast web liquidsoap -h ladspa.plugin
```

We have modified the MK Pascal script to work with AzuraCast.

You can check out the full script here that you can include in your Liquidsoap configuration via the `Utilities -> Edit Liquidsoap Configuration` page of your station.

Scrobbling to Last.fm

Liquidsoap has a native Last.fm extension builtin. To use it, you need to obtain an "API Key" and "API Secret" from Last.fm's API account page and add some custom code to your Liquidsoap config in the third text box (before `add_skip_command(radio)`).

```
settings.audioscrobbler.api_key := "API_KEY"
settings.audioscrobbler.api_secret := "API_SECRET"

radio = audioscrobbler.submit(
  username="USERNAME",
  password="PASSWORD",
  force=true,
  metadata_preprocessor=(fun(m) -> if m["jingle_mode"] == "true" then [] else
m end),
 radio
)
```

You must change `API_KEY`, `API_SECRET`, `USERNAME`, and `PASSWORD` to reflect the credentials of the account you wish to scrobble to.

While a track is being played on your station it is marked as “Scrobbling now” on the account page of the Last.fm account specified. When the track finishes, it becomes scrobbled. Jingles are not scrobbled. Tracks with no `Artist` or no `Title` are not scrobbled.